CHAPTER 7

String Manipulation

KEY TERMS

string	leading blanks	string concatenation
substring	trailing blanks	delimiter
control characters	vectors of characters	token
white space characters	empty string	

CONTENTS

7.1 Creating String Variables 237

7.2 Operations on Strings 240

7.3 The "is" Functions for Strings 255

7.4 Converting Between String and Number Types 256

Summary 259

Common Pitfalls 259

Programming Style Guidelines 259

A ***string*** in the MATLAB® software consists of any number of characters and is contained in single quotes. Actually, strings are vectors in which every element is a single character, which means that many of the vector operations and functions that we have already seen work with strings.

MATLAB also has many built-in functions that are written specifically to manipulate strings. In some cases strings contain numbers, and it is useful to convert from strings to numbers and vice versa; MATLAB has functions to do this as well.

There are many applications for using strings, even in fields that are predominantly numerical. For example, when data files consist of combinations of numbers and characters, it is often necessary to read each line from the file as a string, break the string into pieces, and convert the parts that contain numbers to number variables that can be used in computations. In this chapter the string manipulation techniques necessary for this will be introduced, and applications in file input/output will be demonstrated in Chapter 9.

7.1 CREATING STRING VARIABLES

A string consists of any number of characters (including, possibly, none). The following are examples of strings:

```
''
' '
'x'
'cat'
'Hello there'
'123'
```


MATLAB®. http://dx.doi.org/10.1016/B978-0-12-804525-1.00007-6
© 2017 Elsevier Inc. All rights reserved.

A ***substring*** is a subset or part of a string. For example, 'there' is a substring within the string 'Hello there'.

Characters include letters of the alphabet, digits, punctuation marks, white space, and control characters. ***Control characters*** are characters that cannot be printed, but accomplish a task (e.g., a backspace or tab). ***White space characters*** include the space, tab, newline (which moves the cursor down to the next line), and carriage return (which moves the cursor to the beginning of the current line). ***Leading blanks*** are blank spaces at the beginning of a string, for example, ' hello', and ***trailing blanks*** are blank spaces at the end of a string.

There are several ways by which string variables can be created. One is by using an assignment statement:

```
>> word = 'cat';
```

Another method is to read into a string variable. Recall that to read into a string variable using the **input** function, the second argument 's' must be included:

```
>> strvar = input('Enter a string: ', 's')
Enter a string: xyzabc
strvar =
xyzabc
```

If leading or trailing blanks are typed by the user, these will be stored in the string. For example, in the following, the user entered four blanks and then 'xyz':

```
>> s = input('Enter a string: ','s')
Enter a string:     xyz
s =
    xyz
```

7.1.1 Strings as Vectors

Strings are treated as ***vectors of characters*** or, in other words, a vector in which every element is a single character, so many vector operations can be performed. For example, the number of characters in a string can be found using the **length** function:

```
>> length('cat')
ans =
     3

>> length(' ')   % one space
ans =
     1

>> length('')   % empty string
ans =
     0
```

Note

There is a difference between an ***empty string***, which has a length of 0, and a string consisting of a blank space, which has a length of 1.

Expressions can refer to an individual element (a character within the string), or a subset of a string, or a transpose of a string:

```
>> mystr = 'Hi';
>> mystr(1)
ans =
H

>> mystr'
ans =
H
i

>> sent = 'Hello there';
>> length(sent)
ans =
     11
>> sent(4:8)
ans =
lo th
```

Note
A blank space in a string is a valid character within the string.

A character matrix can be created that consists of strings in every row. The following is created as a column vector of strings, but the end result is that it is a matrix in which every element is a character:

```
>> wordmat = ['Hello';'Howdy']
wordmat =
Hello
Howdy
>> size(wordmat)
ans =
    2      5
```

This created a *2 x 5* matrix of characters.

With a character matrix we can refer to an individual element (a single character) or an individual row (one of the strings):

```
>> wordmat(2,4)
ans =
d

>> wordmat(1,:)
ans =
Hello
```

As rows within a matrix must always be of the same length, the shorter strings must be padded with blanks so that all strings have the same length; otherwise, an error will occur.

```
>> greetmat = ['Hello'; 'Goodbye']     % Incorrect
Error using vertcat
Dimensions of matrices being concatenated are not consistent.
```

```
>> greetmat = ['Hello '; 'Goodbye']
greetmat  =
Hello
Goodbye
>> size(greetmat)
ans  =
         2     7
```

PRACTICE 7.1

Prompt the user for a string. Print the length of the string and also the first and last characters in the string. Make sure that this works regardless of what the user enters.

7.2 OPERATIONS ON STRINGS

MATLAB has many built-in functions that work with strings. Some of the string manipulation functions that perform the most common operations will be described here.

7.2.1 Concatenation

String concatenation means to join strings together. Of course, as strings are just vectors of characters, the method of concatenating vectors also works for strings. For example, to create one long string from two strings, it is possible to join them by putting them in square brackets:

```
>> first = 'Bird';
>> last = 'house';
>> [first last]
ans  =
Birdhouse
```

Note that the variable names (or strings) must be separated by a blank space in the brackets, but there is no space in between the strings when they are concatenated.

The function **strcat** concatenates horizontally, meaning that it creates one longer string from the inputs.

```
>> first = 'Bird';
>> last = 'house';
>> strcat(first,last)
ans  =
Birdhouse
```

If there are leading or trailing blanks in the strings, there is a difference between these two methods of concatenating. The method of using the square brackets will concatenate all of the characters in the strings, including all leading and trailing blanks.

```
>> str1 = 'xxx   ';
>> str2 = '   yyy';
>> [str1 str2]
ans =
xxx      yyy

>> length(ans)
ans =
    12
```

The **strcat** function, however, will remove trailing blanks (but not leading blanks) from strings before concatenating. Note that in these examples, the trailing blanks from *str1* are removed, but the leading blanks from *str2* are not:

```
>> strcat(str1,str2)
ans =
xxx   yyy

>> length(ans)
ans =
     9

>> strcat(str2,str1)
ans =
   yyyxxx

>> length(ans)
ans =
     9
```

We have seen already that the **char** function can be used to convert from an ASCII code to a character. The **char** function can also be used to concatenate vertically, meaning that it will create a column vector of strings (or, in other words, create a matrix of characters). When using the **char** function to create a matrix, it will automatically pad the strings in the rows with trailing blanks as necessary so that they are all of the same length.

```
>> clear greetmat
>> greetmat = char('Hello','Goodbye')
greetmat =
Hello
Goodbye
>> size(greetmat)
ans =
     2     7
```

PRACTICE 7.2

Create a variable that stores the word 'northeast'. From this, create two separate variables *v1* and *v2* that store 'north' and 'east'. Then, create a matrix consisting of the values of *v1* and *v2* in separate rows.

7.2.2 Creating Customized Strings

There are several built-in functions that create customized strings, including **blanks** and **sprintf**.

The **blanks** function will create a string consisting of *n* blank characters (which are kind of hard to see!). However, in MATLAB, if the mouse is moved to highlight the result in *ans*, the blanks can be seen.

```
>> b = blanks(4)
b =

>> length(b)
ans  =
      4
```

It is usually most useful to use this function when concatenating strings, and a number of blank spaces is desired in between. For example, this will insert five blank spaces in between the words:

```
>> [first blanks(5) last]
ans  =
Bird      house
```

Displaying the transpose of the string resulting from the **blanks** function can also be used to move the cursor down. In the Command Window it would look like this:

```
>> disp(blanks(4)')



>>
```

This is useful in a script or function to create space in output, and is equivalent to printing the newline character four times.

The **sprintf** function works exactly like the **fprintf** function, *but instead of printing it creates a string*. Here are several examples in which the output is not suppressed so the value of the string variable is shown:

```
>> sent1 = sprintf('The value of pi is %.2f', pi)
sent1  =
The value of pi is 3.14

>> sent2 = sprintf('Some numbers: %5d, %2d', 33, 6)
sent2  =
Some numbers:    33,   6
```

```
>> length(sent2)
ans =
    23
```

In the following example, however, the output of the assignment is suppressed so the string is created including a random integer and stored in the string variable. Then, some exclamation points are concatenated to that string.

```
>> phrase = sprintf('A random integer is %d', ...
                   randi([5,10]));
>> strcat(phrase, '!!!!')
ans =
A random integer is 7!!!!
```

All of the formatting options that can be used in the **fprintf** function can also be used in the **sprintf** function.

7.2.2.1 Applications of Customized Strings: Prompts, Labels, and Arguments to Functions

One very useful application of the **sprintf** function is to create customized strings, including formatting and/or numbers that are not known ahead of time (e.g., entered by the user or calculated). These customized strings can then be passed to other functions, for example, for plot titles or axis labels. For example, assume that a file "expnoanddata.dat" stores an experiment number, followed by the experiment data. In this case the experiment number is "123," and then the rest of the file consists of the actual data.

```
123   4.4   5.6   2.5   7.2   4.6   5.3
```

The following script would load these data and plot them with a title that includes the experiment number.

plotexpno.m

```
% This script loads a file that stores an experiment number
% followed by the actual data.  It plots the data and puts
% the experiment # in the plot title

load expnoanddata.dat
experNo = expnoanddata(1);
data = expnoanddata(2:end);
plot(data,'ko')
xlabel('Sample #')
ylabel('Weight')
title(sprintf('Data from experiment %d', experNo))
```

The script loads all numbers from the file into a row vector. It then separates the vector; it stores the first element, which is the experiment number, in a variable *experNo*, and the rest of the vector in a variable *data* (the rest being from the second element to the end). It then plots the data, using **sprintf** to create the title, which includes the experiment number as seen in Fig. 7.1.

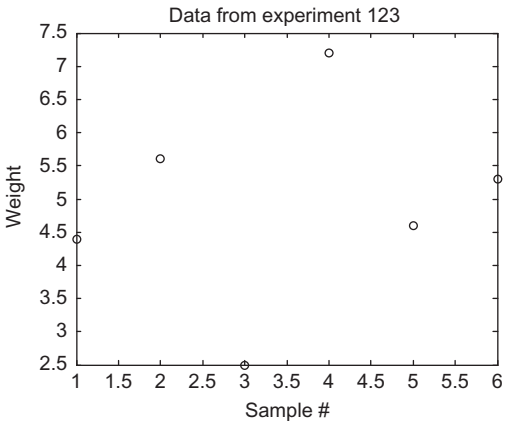


FIGURE 7.1
Customized title in plot using **sprintf**.

PRACTICE 7.3

In a loop, create and print strings with file names "file1.dat," "file2.dat," and so on for file numbers 1 through 5.

QUICK QUESTION!

How could we use the **sprintf** function to customize prompts for the **input** function?
Answer: For example, if it is desired to have the contents of a string variable printed in a prompt, **sprintf** can be used:

```
>> username = input('Please enter your name: ', 's');
Please enter your name: Bart

>> prompt = sprintf('%s, Enter your id #: ',username);
>> id_no = input(prompt)
Bart, Enter your id #: 177
id_no =
   177
```

Another way of accomplishing this (in a script or function) would be:

```
fprintf('%s, Enter your id #: ',username);
id_no = input('');
```

Note that the calls to the **sprintf** and **fprintf** functions are identical except that the **fprintf** prints (so there is no need for a prompt in the **input** function), whereas the **sprintf** creates a string that can then be displayed by the **input** function. In this case using **sprintf** seems cleaner than using **fprintf** and then having an empty string for the prompt in **input**.

As another example, the following program prompts the user for endpoints (x_1, y_1) and (x_2, y_2) of a line segment, and calculates the midpoint of the line segment, which is the point (x_m, y_m). The coordinates of the midpoint are found by:

$$x_m = \frac{1}{2}(x_1 + x_2) \qquad y_m = \frac{1}{2}(y_1 + y_2)$$

The script *midpoint* calls a function *entercoords* to separately prompt the user for the x and y coordinates of the two endpoints, calls a function *findmid* twice to calculate separately the x and y coordinates of the midpoint, and then prints this midpoint. When the program is executed, the output looks like this:

```
>> midpoint
Enter the x coord of the first endpoint: 2
Enter the y coord of the first endpoint: 4
Enter the x coord of the second endpoint: 3
Enter the y coord of the second endpoint: 8
The midpoint is (2.5, 6.0)
```

In this example, the word 'first' or 'second' is passed to the *entercoords* function so that it can use whichever word is passed in the prompt. The prompt is customized using **sprintf**.

midpoint.m

```
% This program finds the midpoint of a line segment

[x1, y1] = entercoords('first');
[x2, y2] = entercoords('second');

midx = findmid(x1,x2);
midy = findmid(y1,y2);

fprintf('The midpoint is (%.1f, %.1f)\n',midx,midy)
```

entercoords.m

```
function [xpt, ypt] = entercoords(word)
% entercoords reads in & returns the coordinates of
%    the specified endpoint of a line segment
% Format: entercoords(word) where word is 'first'
%         or 'second'

prompt = sprintf('Enter the x coord of the %s endpoint: ', ...
      word);
xpt = input(prompt);

prompt = sprintf('Enter the y coord of the %s endpoint: ', ...
      word);
ypt = input(prompt);
end
```

findmid.m

```
function mid = findmid(pt1,pt2)
% findmid calculates a coordinate (x or y) of the
%   midpoint of a line segment
% Format: findmid(coord1, coord2)

mid = 0.5 * (pt1 + pt2);
end
```

7.2.3 Removing White Space Characters

MATLAB has functions that will remove trailing blanks from the end of a string and/or leading blanks from the beginning of a string.

The **deblank** function will remove blank spaces from the end of a string. For example, if some strings are padded in a character matrix so that all are of the same length, it is frequently desired to then remove those extra blank spaces to use the string in its original form.

```
>> names = char('Sue', 'Cathy','Xavier')
names  =
Sue
Cathy
Xavier

>> name1 = names(1,:)
name1  =
Sue
>> length(name1)
ans  =
     6
>> name1 = deblank(name1);
>> length(name1)
ans  =
     3
```

Note

The **deblank** function only removes trailing blanks from a string, not leading blanks.

The **strtrim** function will remove both leading and trailing blanks from a string, but not blanks in the middle of the string. In the following example, the three blanks in the beginning and four blanks in the end are removed, but not the two blanks in the middle. Highlighting the result in the Command Window with the mouse would show the blank spaces.

```
>> strvar = [blanks(3) 'xx' blanks(2) 'yy' blanks(4)]
strvar  =
   xx  yy
```

```
>> length(strvar)
ans  =
    13
>> strtrim(strvar)
ans  =
xx  yy
>> length(ans)
ans  =
     6
```

7.2.4 Changing Case

MATLAB has two functions that convert strings to all uppercase letters, or lowercase, called **upper** and **lower**.

```
>> mystring = 'AbCDEfgh';
>> lower(mystring)
ans  =
abcdefgh
>> upper(ans)
ans  =
ABCDEFGH
```

PRACTICE 7.4

Assume that these expressions are typed sequentially in the Command Window. Think about it, write down what you think the results will be, and then verify your answers by actually typing them.

```
lnstr = '1234567890';
mystr = '   abc    xy';
newstr = strtrim(mystr)
length(newstr)
upper(newstr(1:3))
sprintf('Number is %4.1f', 3.3)
```

7.2.5 Comparing Strings

There are several functions that compare strings and return **logical true** if they are equivalent, or **logical false** if not. The function **strcmp** compares strings, character by character. It returns **logical true** if the strings are completely identical (which infers that they must also be of the same length), or **logical false** if the strings are not of the same length or any corresponding characters are not

identical. Note that for strings, these functions are used to determine whether strings are equal to each other or not, not the equality operator ==. Here are some examples of these comparisons:

```
>> word1 = 'cat';
>> word2 = 'car';
>> word3 = 'cathedral';
>> word4 = 'CAR';
>> strcmp(word1,word3)
ans  =
     0
>> strcmp(word1,word1)
ans  =
     1
>> strcmp(word2,word4)
ans  =
     0
```

The function **strncmp** compares only the first *n* characters in strings and ignores the rest. The first two arguments are the strings to compare and the third argument is the number of characters to compare (the value of *n*).

```
>> strncmp(word1,word3,3)
ans  =
     1
```

QUICK QUESTION!

How can we compare strings, ignoring whether the characters in the string are uppercase or lowercase?

Answer: See the following Programming Concept and Efficient Method.

THE PROGRAMMING CONCEPT

Ignoring the case when comparing strings can be done by changing all characters in the strings to either upper- or lowercase, for example, in MATLAB using the **upper** or **lower** function:

```
>> strcmp(upper(word2),upper(word4))
ans  =
     1
```

THE EFFICIENT METHOD

The function **strcmpi** compares the strings but ignores the case of the characters.

```
>> strcmpi(word2,word4)
ans  =
     1
```

There is also a function **strncmpi,** which compares *n* characters, ignoring the case.

7.2.6 Finding, Replacing, and Separating Strings

There are functions that find and replace strings, or parts of strings, within other strings and functions that separate strings into substrings.

The function **strfind** receives two strings as input arguments. The general form is **strfind(string, substring)**; it finds all occurrences of the substring within the string, and returns the subscripts of the beginning of the occurrences. The substring can consist of one character, or any number of characters. If there is more than one occurrence of the substring within the string, **strfind** returns a vector with all indices. Note that what is returned is the index of the beginning of the substring.

```
>> strfind('abcde', 'd')
ans  =
     4
>> strfind('abcde', 'bc')
ans  =
     2
>> strfind('abcdeabcdedd', 'd')
ans  =
     4     9    11    12
```

If there are no occurrences, the empty vector is returned.

```
>> strfind('abcdeabcde','ef')
ans  =
     []
```

QUICK QUESTION!

How can you find how many blanks there are in a string (e.g., 'how are you')?

Answer: The **strfind** function will return an index for every occurrence of a substring within a string, so the result is a vector of indices. The **length** of this vector of indices would be the number of occurrences. For example, the following finds the number of blank spaces in the variable *phrase*.

```
>> phrase = 'Hello, and how are you doing?';
>> length(strfind(phrase,' '))
ans =
     5
```

If it is desired to get rid of any leading and trailing blanks first (in case there are any), the **strtrim** function would be used first.

```
>> phrase  = '  Well, hello there!  ';
>> length(strfind(strtrim(phrase),' '))
ans  =
     2
```

Let's expand this and write a script that creates a vector of strings that are phrases. The output is not suppressed so that the strings can be seen when the script is executed. It loops through this vector and passes each string to a function *countblanks*. This function counts the number of blank spaces in the string, not including any leading or trailing blanks.

phraseblanks.m

```
% This script creates a column vector of phrases
% It loops to call a function to count the number
%  of blanks in each one and prints that

phrasemat = char('Hello and how are you?', ...
  'Hi there everyone!', 'How is it going?', 'Whazzup?')
[r c]  = size(phrasemat);

for i = 1:r
    % Pass each row (each string) to countblanks function
    howmany = countblanks(phrasemat(i,:));
    fprintf('Phrase %d had %d blanks\n',i,howmany)
end
```

countblanks.m

```
function num = countblanks(phrase)
% countblanks returns the # of blanks in a trimmed string
% Format: countblanks(string)

num = length(strfind(strtrim(phrase), ' '));
end
```

For example, running this script would result in:

```
>> phraseblanks
phrasemat =
Hello and how are you?
Hi there everyone!
How is it going?
Whazzup?

Phrase 1 had 4 blanks
Phrase 2 had 2 blanks
Phrase 3 had 3 blanks
Phrase 4 had 0 blanks
```

The function **strrep** finds all occurrences of a substring within a string, and replaces them with a new substring. The order of the arguments matters. The format is

```
strrep(string, oldsubstring, newsubstring)
```

The following replaces all occurrences of the substring 'e' with the substring 'x':

```
>> strrep('abcdeabcde','e','x')
ans =
abcdxabcdx
```

All strings can be of any length, and the lengths of the old and new substrings do not have to be the same. If the old substring is not found, nothing is changed in the original string.

In addition to the string functions that find and replace, there is a function that separates a string into two substrings. The **strtok** function breaks a string into two pieces; it can be called several ways. The function receives one string as an input argument. It looks for the first ***delimiter***, which is a character or set of characters that act as a separator within the string.

By default, the delimiter is any white space character. The function returns a ***token*** that is the beginning of the string, up to (but not including) the first delimiter. It also returns the rest of the string, which includes the delimiter. Assigning the returned values to a vector of two variables will capture both of these. The format is

```
[token, rest] = strtok(string)
```

where *token* and *rest* are variable names. For example,

```
>> sentence1 = 'Hello there';
>> [word, rest] = strtok(sentence1)
word =
Hello
rest =
 there

>> length(word)
ans =
     5
>> length(rest)
ans =
     6
```

Note that the rest of the string includes the blank space delimiter.

Alternate delimiters can be defined. The format

```
[token, rest] = strtok(string, delimeters)
```

returns a token that is the beginning of the string, up to the first character contained within the delimiters string, and also the rest of the string. In the following example, the delimiter is the character 'l'.

```
>> [word, rest] = strtok(sentence1,'l')
word =
He
rest =
llo there
```

Leading delimiter characters are ignored, whether it is the default white space or a specified delimiter. For example, the leading blanks are ignored here:

```
>> [firstpart, lastpart] = strtok(' materials science')
firstpart =
materials

lastpart =
 science
```

QUICK QUESTION!

What do you think **strtok** returns if the delimiter is not in the string?

Answer: The first result returned will be the entire string, and the second will be the empty string.

```
>> [first, rest] = strtok('ABCDE')
first =
ABCDE

rest =
   Empty string: 1-by-0
```

PRACTICE 7.5

Think about what would be returned by the following sequence of expressions and statements, and then type them into MATLAB to verify your results.

```
dept = 'Electrical';
strfind(dept,'e')

strfind(lower(dept),'e')

phone_no = '703-987-1234';
[area_code, rest] = strtok(phone_no,'-')

rest = rest(2:end)

strcmpi('Hi','HI')
```

QUICK QUESTION!

The function **date** returns the current date as a string (e.g., '10-Dec-2015'). How could we write a function to return the day, month, and year as separate output arguments?

Answer: We could use **strrep** to replace the '-' characters with blanks and then use **strtok** with the blank as the default delimiter to break up the string (twice) or, more simply, we could just use **strtok** and specify the '-' character as the delimiter.

separatedate.m

```
function [todayday, todaymo, todayyr] = separatedate
% separatedate separates the current date into day,
% month, and year
% Format: separatedate or separatedate()

[todayday, rest] = strtok(date,'-');
[todaymo, todayyr] = strtok(rest,'-');
todayyr = todayyr(2:end);
end
```

As we need to separate the string into three parts, we need to use the **strtok** function twice. The first time the string is separated into '10' and '-Dec-2015' using **strtok**. Then, the second string is separated into 'Dec' and '-2015' using **strtok**. (As leading delimiters are ignored the second '-' is found as the delimiter in '-Dec-2015'.) Finally, we need to remove the '-' from the string '-2015'; this can be done by just indexing from the second character to the end of the string.

An example of calling this function follows:

```
>> [d, m, y] = separatedate
d =
10
m =
Dec
y =
2015
```

Note that no input arguments are passed to the *separatedate* function; instead, the **date** function returns the current date as a string. Also, note that all three output arguments are strings.

7.2.7 Evaluating a String

The function **eval** is used to evaluate a string. If the string contains a call to a function, then that function call is executed. For example, in the following, the string 'plot(x)' is evaluated to be a call to the **plot** function, and it produces the plot shown in Fig. 7.2.

```
>> x = [2 6 8 3];
>> eval('plot(x)')
```

The **eval** function is frequently used when input is used to create a customized string. In the following example, the user chooses the type of plot to use for some quiz grades. The string that the user enters (in this case, 'bar') is concatenated with the string '(x)' to create the string 'bar(x)'; this is then evaluated as a call to the **bar** function as seen in Fig. 7.3. The name of the plot type is also used in the title.

```
>> x = [9 7 10 9];
>> whatplot = input('What type of plot?: ', 's');
What type of plot?: bar
>> eval([whatplot '(x)'])
>> title(whatplot)
>> xlabel('Student #')
>> ylabel('Quiz Grade')
```

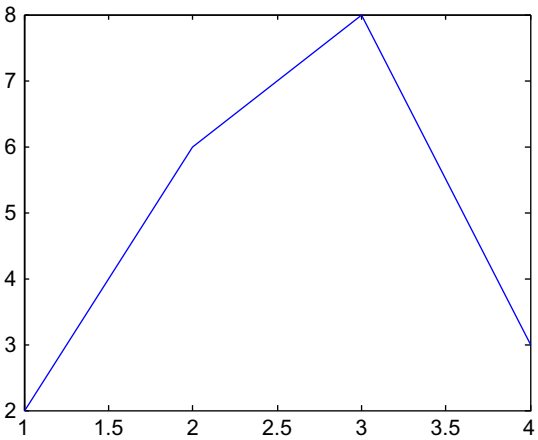


FIGURE 7.2
Plot type passed to the **eval** function.

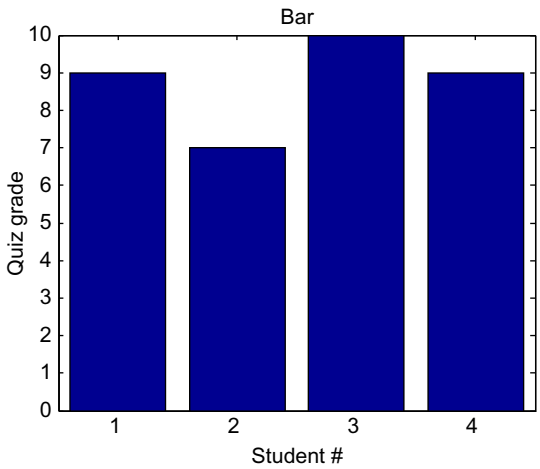


FIGURE 7.3
Plot type entered by the user.

PRACTICE 7.6

Create an *x* vector. Prompt the user for 'sin', 'cos', or 'tan' and create a string with that function of *x* (e.g., 'sin(x)' or 'cos(x)'). Use **eval** to create a *y* vector using the specified function.

The **eval** function is very powerful, but it is usually more efficient to avoid using it.

7.3 THE "IS" FUNCTIONS FOR STRINGS

There are several "is" functions for strings, which return **logical true** or **false**. The function **isletter** returns **logical true** for every character in a string if the character is a letter of the alphabet or **false** if not. The function **isspace** returns **logical true** for every character that is a white space character.

```
>> isletter('EK127')
ans =
     1     1     0     0     0

>> isspace('a b')
ans =
     0     1     0
```

The **isstrprop** function determines whether the characters in a string are in a category specified by a second argument. For example, the following tests to see whether or not the characters are alphanumeric; all are except for the dot '.'.

```
>> isstrprop('AB123.4','alphanum')
ans =
     1     1     1     1     1     0     1
```

The **ischar** function will return **logical true** if the vector argument is a character vector (in other words, a string), or **logical false** if not.

```
>> vec = 'EK127';
>> ischar(vec)
ans =
     1

>> vec = 3:5;
>> ischar(vec)
ans =
     0
```

7.4 CONVERTING BETWEEN STRING AND NUMBER TYPES

MATLAB has several functions that convert numbers to strings in which each character element is a separate digit and vice versa.

Note
These are different from the functions such as **char** and **double** that convert characters to ASCII equivalents and vice versa.

To convert numbers to strings, MATLAB has the functions **int2str** for integers and **num2str** for real numbers (which also works with integers). The function **int2str** would convert, for example, the integer 38 to the string '38'.

```
>> num = 38;
num  =
     38
>> s1 = int2str(num)
s1  =
38

>> length(num)
ans  =
      1
>> length(s1)
ans  =
      2
```

The variable *num* is a scalar that stores one number, whereas *s1* is a string that stores two characters, '3' and '8'.

Even though the result of the first two assignments is "38," note that the indentation in the Command Window is different for the number and the string.

The **num2str** function, which converts real numbers, can be called in several ways. If only the real number is passed to the **num2str** function, it will create a string that has four decimal places, which is the default in MATLAB for displaying real numbers. The precision can also be specified (which is the number of digits), and format strings can also be passed, as shown in the following:

```
>> str2 = num2str(3.456789)
str2  =
3.4568
>> length(str2)
ans  =
      6

>> str3 = num2str(3.456789,3)
str3  =
3.46
```

```
>> str = num2str(3.456789,'%6.2f')
str  =
3.46
```

Note that in the last example, MATLAB removed the leading blanks from the string.

The functions **str2double** and **str2num** do the reverse; they takes a string in which number(s) are stored and converts them to the type **double**:

```
>> num = str2double('123.456')
num  =
   123.4560
```

If there is a string in which there are numbers separated by blanks, the **str2num** function will convert this to a vector of numbers (of the default type **double**). For example,

```
>> mystr = '66 2 111';
>> numvec = str2num(mystr)
numvec =
    66     2   111

>> sum(numvec)
ans  =
   179
```

The **str2double** function is a better function to use in general than **str2num**, but it can only be used when a scalar is passed; it would not work, for example, for the variable *mystr* above.

PRACTICE 7.7

Think about what would be returned by the following sequence of expressions and statements, and then type them into MATLAB to verify your results.

```
vec = 'yes or no';
isspace(vec)

all(isletter(vec) ~= isspace(vec))

ischar(vec)

nums = [33 1.5];
num2str(nums)

nv = num2str(nums)

sum(nums)
```

QUICK QUESTION!

Let's say that we have a string that consists of an angle followed by either 'd' for degrees or 'r' for radians. For example, it may be a string entered by the user:

```
degrad = input('Enter angle and d/r: ', 's');
Enter angle and d/r: 54r
```

How could we separate the string into the angle and the character, and then get the sine of that angle using either **sin** or **sind**, as appropriate (**sin** for radians or **sind** for degrees)?

Answer: First, we could separate this string into its two parts:

```
>> angle = degrad(1:end-1)
angle =
54
>> dorr = degrad(end)
dorr =
r
```

Then, using an **if-else** statement, we would decide whether to use the **sin** or **sind** function, based on the value of the variable *dorr*. Let's assume that the value is 'r' so we want to use **sin**. The variable *angle* is a string so the following would not work:

```
>> sin(angle)
Undefined function 'sin' for input arguments
of type 'char'.
```

Instead, we could either use **str2double** to convert the string to a number. A complete script to accomplish this is shown here.

angleDorR.m

```
% Prompt the user for angle and 'd' for degrees
% or 'r' for radians; print the sine of the angle

% Read in the response as a string and then
% separate the angle and character
degrad = input('Enter angle and d/r: ', 's');
angle = degrad(1:end-1);
dorr = degrad(end);

% Error-check to make sure user enters 'd' or 'r'
while dorr ~= 'd' && dorr ~= 'r'
   disp('Error! Enter d or r with the angle.')
   degrad = input('Enter angle and d/r: ', 's');
   angle = degrad(1:end-1);
   dorr = degrad(end);
end
%  Convert angle to number
anglenum = str2double(angle);
fprintf('The sine of %.1f ', anglenum)
% Choose sin or sind function
if dorr == 'd'
   fprintf('degrees is %.3f.\n', sind(anglenum))
else
   fprintf('radians is %.3f.\n', sin(anglenum))
end
```

```
>> angleDorR
Enter angle and d/r: 3.1r
The sine of 3.1 radians is 0.042.
```

```
>> angleDorR
Enter angle and d/r: 53t
Error! Enter d or r with the angle.
Enter angle and d/r: 53d
The sine of 53.0 degrees is 0.799.
```

Explore Other Interesting Features

In many of the search and replace functions, search patterns can be specified which use *regular expressions*. Use **help** to find out about these patterns.

Explore the **sscanf** function, which reads data from a string.

Explore the **strjust** function, which justifies a string.

Explore the **mat2str** function, to convert from a matrix to a string.

Explore the properties that can be examined with the **isstrprop** function.

Investigate why the string compare functions are used to compare strings, rather than the equality operator. ■

SUMMARY

COMMON PITFALLS

- Putting arguments to **strfind** in incorrect order.
- Trying to use == to compare strings for equality, instead of the **strcmp** function (or its variations).
- Confusing **sprintf** and **fprintf**. The syntax is the same, but **sprintf** creates a string whereas **fprintf** prints.
- Trying to create a vector of strings with varying lengths (the easiest way is to use **char** which will pad with extra blanks automatically).
- Forgetting that when using **strtok**, the second argument returned (the "rest" of the string) contains the delimiter.
- When breaking a string into pieces, forgetting to convert the numbers in the strings to actual numbers that can then be used in calculations.

PROGRAMMING STYLE GUIDELINES

- Trim trailing blanks from strings that have been stored in matrices before using.
- Make sure that the correct string comparison function is used, for example, **strcmpi** if ignoring case is desired.

MATLAB Functions and Commands			
strcat	lower	strrep	isstrprop
blanks	strcmp	strtok	ischar
sprintf	strncmp	date	int2str
deblank	strcmpi	eval	num2str
strtrim	strncmpi	isletter	str2double
upper	strfind	isspace	str2num

Exercises

1. A file name is supposed to be in the form *filename.ext*. Write a function that will determine whether a string is in the form of a name followed by a dot followed by a three-character extension, or not. The function should return 1 for **logical true** if it is in that form, or 0 for **false** if not.
2. The following script calls a function *getstr* that prompts the user for a string, error-checking until the user enters something (the error would occur if the user just hits the Enter key without any other characters first). The script then prints the length of the string. Write the *getstr* function.

```
thestring = getstr();
fprintf('Thank you, your string is %d characters long\n', ...
    length(thestring))
```

3. Write a script that will, in a loop, prompt the user for four course numbers. Each will be a string of length 5 of the form 'CS101'. These strings are to be stored in a character matrix.
4. Write a function that will generate two random integers, each in the inclusive range from 10 to 30. It will then return a string consisting of the two integers joined together, for example, if the random integers are 11 and 29, the string that is returned will be '1129'.
5. Write a script that will create *x* and *y* vectors. Then, it will ask the user for a color ('red,' 'blue,' or 'green') and for a plot style (circle or star). It will then create a string *pstr* that contains the color and plot style, so that the call to the **plot** function would be: **plot(x,y,pstr)**. For example, if the user enters 'blue' and '*', the variable *pstr* would contain 'b*'.
6. Assume that you have the following function and that it has not yet been called.

strfunc.m

```
function strfunc(instr)
persistent mystr
if isempty(mystr)
    mystr = '';
end
mystr = strcat(instr,mystr);
fprintf('The string is %s\n',mystr)
end
```

What would be the result of the following sequential expressions?

```
strfunc('hi')
strfunc('hello')
```

7. Explain in words what the following function accomplishes (not step-by-step, but what the end result is).

dostr.m

```
function out = dostr(inp)
persistent str
[w, r] = strtok(inp);
str = strcat(str,w);
out = str;
end
```

8. Write a function that will receive a name and department as separate strings and will create and return a code consisting of the first two letters of the name and the last two letters of the department. The code should be upper-case letters. For example,

```
>> namedept('Robert','Mechanical')
ans =
ROAL
```

9. Write a function "createUniqueName" that will create a series of unique names. When the function is called, a string is passed as an input argument. The functions adds an integer to the end of the string, and returns the resulting string. Every time the function is called, the integer that it adds is incremented. Here are some examples of calling the function:

```
>> createUniqueName('hello')
ans =
hello1
>> varname = createUniqueName('variable')
varname =
variable2
```

10. What does the **blanks** function return when a 0 is passed to it? A negative number? Write a function *myblanks* that does exactly the same thing as the **blanks** function, using the programming method. Here are some examples of calling it:

```
>> fprintf('Here is the result:%s!\n', myblanks(0))
Here is the result:!

>> fprintf('Here is the result:%s!\n', myblanks(7))
Here is the result:        !
```

11. Write a function that will prompt the user separately for a filename and extension and will create and return a string with the form 'filename.ext'.
12. Write a function that will receive one input argument, which is an integer *n*. The function will prompt the user for a number in the range from 1 to *n* (the actual value of *n* should be printed in the prompt) and return the user's input. The function should error-check to make sure that the user's input is in the correct range.
13. Write a script that will generate a random integer, ask the user for a field width, and print the random integer with the specified field width. The script will use **sprintf** to create a string such as 'The # is %4d\n' (for e.g., if, the user entered 4

for the field width) which is then passed to the **fprintf** function. To print (or create a string using **sprintf**) either the % or \ character, there must be two of them in a row.

14. Write a function called *plotsin* that will graphically demonstrate the difference in plotting the sin function with a different number of points in the range from 0 to 2 Π. The function will receive two arguments, which are the number of points to use in two different plots of the sin function. For example, the following call to the function:

```
>> plotsin(5,30)
```

will result in Fig. 7.4 in which the first plot has five points altogether in the range from 0 to 2 Π, inclusive, and the second has 30:

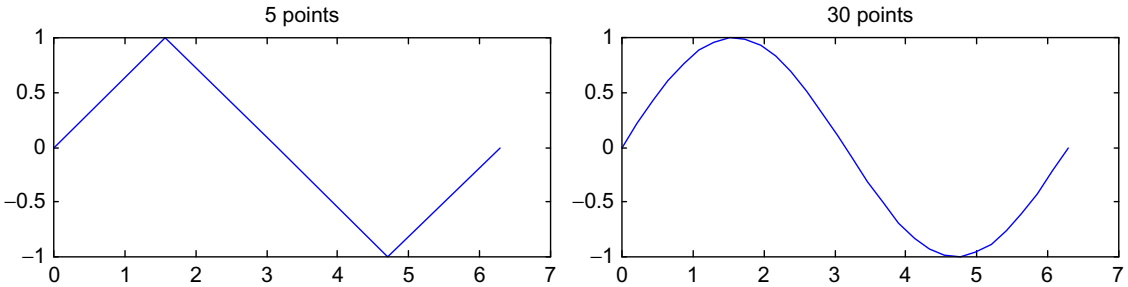


FIGURE 7.4
Subplot with sin.

15. If the strings passed to **strfind** are of the same length, what are the only two possible results that could be returned?
16. Vectorize this:

```
while mystrn(end) == ' '  % Note one space in quotes
     mystrn = mystrn(1:end-1);
end
```

17. Vectorize this:

```
loc = findstr(sentence, ' '); % one blank space
where = loc(1);
first = sentence(1:where-1);
last = sentence(where:end);
```

18. Vectorize this:

```
vec = [];
for i = 1:8
   vec = [ vec ' ']; % one blank space
end
vec   % just for display
```

19. Vectorize this:

```
if length(str1) ~= length(str2)
    outlog = false;
else
    outlog = true;
    for i=1:length(str1)
        if str1(i)   ~= str2(i)
            outlog = false;
        end
    end
end
outlog % Just to display the value
```

20. Write a function *nchars* that will create a string of *n* characters, without using any loops or selection statements.

```
>> nchars('*', 6)
ans  =
******
```

21. Write a function that will receive two input arguments: a character matrix that is a column vector of strings, and a string. It will loop to look for the string within the character matrix. The function will return the row number in which the string is found if it is in the character matrix, or the empty vector if not. Use the programming method.
22. Write a function *rid_multiple_blanks* that will receive a string as an input argument. The string contains a sentence that has multiple blank spaces in between some of the words. The function will return the string with only one blank in between words. For example,

```
>> mystr = 'Hello  and how  are  you?';
>> rid_multiple_blanks(mystr)
ans =
Hello and how are you?
```

23. Words in a string variable are separated by right slashes (/) instead of blank spaces. Write a function *slashtoblank* that will receive a string in this form and will return a string in which the words are separated by blank spaces. This should be general and work regardless of the value of the argument. No loops allowed in this function; the built-in string function(s) must be used.
24. Two variables store strings that consist of a letter of the alphabet, a blank space, and a number (in the form 'R 14.3'). Write a script that would initialize two such variables. Then, use string manipulating functions to extract the numbers from the strings and add them together.

Cryptography, or encryption, is the process of converting plaintext (e.g., a sentence or paragraph), into something that should be unintelligible, called the ciphertext. The reverse process is code-breaking, or cryptanalysis, which relies on searching the

encrypted message for weaknesses and deciphering it from that point. Modern security systems are heavily reliant on these processes.

25. In cryptography, the intended message sometimes consists of the first letter of every word in a string. Write a function *crypt* that will receive a string with the encrypted message and return the message.

```
>> estring = 'The early songbird tweets';
>> m  = crypt(estring)
m  =
Test
```

26. Using the functions **char** and **double**, one can shift words. For example, one can convert from lower case to upper case by subtracting 32 from the character codes:

```
>> orig = 'ape';
>> new = char(double(orig)-32)
new  =
APE

>> char(double(new)+32)
ans  =
ape
```

We've "encrypted" a string by altering the character codes. Figure out the original string. Try adding and subtracting different values (do this in a loop) until you decipher it:

```
Jmkyvih$mx$syx$}ixC
```

27. Load files named *file1.dat*, *file2.dat*, and so on in a loop. To test this, create just two files with these names in your Current Folder first.
28. Either in a script or in the Command Window, create a string variable that stores a string in which numbers are separated by the character 'x', for example, '12x3x45x2'. Create a vector of the numbers, and then get the sum (Note: For the example given it would be 62 but the solution should be general).
29. Create the following two variables:

```
>> var1 = 123;
>> var2 = '123';
```

Then, add 1 to each of the variables. What is the difference?

30. The built-in **clock** function returns a vector with six elements representing the year, month, day, hours, minutes, and seconds. The first five elements are integers whereas the last is a **double** value, but calling it with **fix** will convert all to integers. The built-in **date** function returns the day, month, and year as a string. For example,

```
>> fix(clock)
ans =
     2013    4    25    14    25    49
>> date
ans =
25-Apr-2013
```

Write a script that will call both of these built-in functions, and then compare results to make sure that the year is the same. The script will have to convert one from a string to a number, or the other from a number to a string in order to compare.

31. Use **help isstrprop** to find out what properties can be tested; try some of them on a string variable.
32. Find out how to pass a vector of integers to **int2str** or real numbers to **num2str**.
33. Write a script that will first initialize a string variable that will store *x* and *y* coordinates of a point in the form 'x 3.1 y 6.4'. Then, use string manipulating functions to extract the coordinates and plot them.
34. Write a function *wordscramble* that will receive a word in a string as an input argument. It will then randomly scramble the letters and return the result. Here is an example of calling the function:

```
>> wordscramble('fantastic')
ans =
safntcait
```

Massive amounts of temperature data have been accumulated and stored in files. To be able to comb through this data and gain insights into global temperature variations, it is often useful to visualize the information.

35. A file called *avehighs.dat* stores for three locations, the average high temperatures for each month for a year (rounded to integers). There are three lines in the file; each stores the location number followed by the 12 temperatures (this format may be assumed). For example, the file might store:

```
432   33 37 42 45 53 72 82 79 66 55 46 41
777   29 33 41 46 52 66 77 88 68 55 48 39
567   55 62 68 72 75 79 83 89 85 80 77 65
```

Write a script that will read these data in and plot the temperatures for the three locations separately in one Figure Window. A **for** loop must be used to accomplish this. For example, if the data are as shown in the previous data block, the Figure Window would appear as in Fig. 7.5. The axis labels and titles should be as shown.

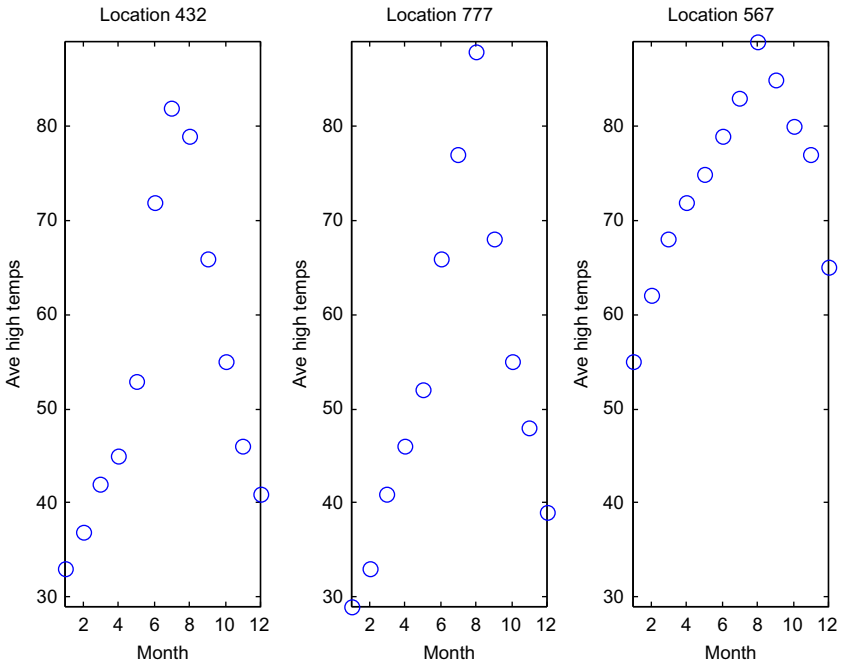


FIGURE 7.5
Subplot to display data from file using a **for** loop.

If you have version R2016a or later, write the script as a live script.